ChainDoRA: Tensor-Train Factorized Weight-Decomposed Low-Rank Adaptation for Parameter-Efficient LLM Fine-Tuning

Ashfak Yeafi[1] Mehedi Hasan[2]
Md Khairul Islam[3]

[1]Department of Electrical and Electronic Engineering, Khulna University of Engineering & Technology, Khulna, Bangladesh
[2]Department of Computer Science and Engineering, Brac University, Dhaka, Bangladesh
[3]Department of Mathematics and Computer Science, Hobart and William Smith Colleges, Geneva, NY, USA

yeafiashfak@gmail.com mehedi.hasan1@g.bracu.ac.bd khairul.robotics@gmail.com

arXiv:submit/8059700 [cs.CV] 9 Sep 2026

Abstract

Parameter-efficient fine-tuning (PEFT) adapts large language models (LLMs) with a small task-specific parameter budget, but the dense low-rank factors used by LoRA and retained in DoRA still scale linearly with the dimensions of each adapted layer. We propose *ChainDoRA*, which preserves DoRA's magnitude–direction decomposition while constructing its directional low-rank factors from a connected Tensor-Train (TT) chain whose boundary rank is the adapter rank and whose independent internal TT rank controls capacity and parameter cost. Under a controlled 15,119-example response-only adaptation setting with LLaMA-7B, ChainDoRA with TT rank 16 achieves 72.30% average accuracy across seven commonsense reasoning benchmarks, compared with 69.88% for LoRA and 69.39% for DoRA. At the same adapter rank, ChainDoRA uses 5.35M trainable parameters versus 56.10M for LoRA and 56.98M for DoRA, a 90.62% reduction relative to DoRA. Ablations over TT rank and adapter placement reveal controllable parameter–accuracy trade-offs, while cross-architecture analysis shows that the structural parameter savings persist across representative 3B–72B model configurations. Code is available at `https://github.com/AshfakYeafi/Chain-DoRA`.

Keywords: parameter-efficient fine-tuning; low-rank adaptation; DoRA; Tensor-Train decomposition; large language models; model adaptation.

1 Introduction

The rapid growth of pretrained language models has made task-specific adaptation increasingly expensive in terms of memory, optimization, and model storage. Conventional full-parameter fine-tuning updates the entire pretrained model and consequently requires maintaining a complete task-specific parameter set. Parameter-efficient fine-tuning (PEFT) addresses this limitation by keeping most or all pretrained parameters frozen while optimizing a comparatively small set of task-specific parameters. Representative approaches include lightweight adapter modules [1], continuous prefix optimization [2], and low-rank reparameterization [3]. Among these methods, Low-Rank Adaptation (LoRA) has become particularly attractive because its learned update can be merged into the pretrained weight at inference time without introducing an additional inference branch.

LoRA parameterizes the update to a frozen weight matrix through two trainable low-rank factors [3]. This substantially reduces the number of optimized parameters relative to full fine-tuning, but its adapter size remains proportional to the input and output dimensions of every adapted linear transformation. Moreover, LoRA modifies the pretrained weight additively without explicitly separating changes in weight scale from changes in orientation. Weight-Decomposed Low-Rank Adaptation (DoRA) addresses the latter issue by decomposing the pretrained weight into magnitude and directional components and applying LoRA to the directional component while learning the magnitude separately [4]. DoRA has demonstrated improved adaptation capability over LoRA across language and vision-language tasks; however, its directional update continues to rely on the same pair of densely parameterized low-rank matrices used by LoRA.

Tensor decompositions offer an alternative mechanism for representing high-dimensional parameters through structured low-dimensional factors. The Tensor-Train (TT) format represents a high-order tensor as a sequence of low-order cores connected through auxiliary TT ranks [5]. TT representations have previously been used to compress neural-network parameters [6], and recent work has extended tensorized parameterizations to PEFT. LoRETTA employs TT representations for ultra-low-parameter LLM adaptation [7], while TT-LoRA explores TT-based parameterization for efficient large-model adaptation [8]. More recent approaches have also investigated weight-

decomposed tensor adaptation and unified TT representations for constructing correlated low-rank factors. These studies indicate that tensor-network structure can substantially reduce the number of directly optimized parameters, but they also highlight the importance of how the tensor representation is integrated into the adaptation mechanism.

Motivated by these observations, we propose *ChainDoRA*, a Tensor-Train-factorized extension of magnitude–direction adaptation. Rather than directly optimizing the two dense low-rank matrices in the directional branch of DoRA, ChainDoRA represents them through contractions of a single connected sequence of trainable TT cores. The input-side cores contract to form $A_{\mathrm{TT}} \in \mathbb{R}^{r \times d_{\mathrm{in}}}$, while the output-side cores form $B_{\mathrm{TT}} \in \mathbb{R}^{d_{\mathrm{out}} \times r}$. The two contractions are connected through a TT bond whose dimension is fixed to the adapter rank r. A separate internal TT rank ρ controls the capacity of the tensorized representation. Consequently, the nominal low-rank adaptation dimension and the internal tensor complexity can be controlled independently.

As illustrated in Fig. 1, the resulting TT-derived update is incorporated into the magnitude–direction decomposition rather than used as an independent additive tensor adapter. This design preserves the row-wise magnitude control of DoRA while replacing its dense directional factors with a structured tensor representation. For the LLaMA-7B configuration studied in this work, adapting the query, key, value, up-projection, and down-projection layers with rank $r = 32$ requires approximately 56.98M trainable parameters with DoRA. ChainDoRA with internal TT rank $\rho = 16$ reduces this number to 5.35M, corresponding to a 90.62% reduction in trainable parameters while retaining the same nominal adapter rank.

We evaluate ChainDoRA under a controlled 15,119-example response-only adaptation setting with LLaMA-7B as the frozen backbone. The evaluation covers seven commonsense reasoning benchmarks: BoolQ, PIQA, SocialIQA, WinoGrande, ARC-Easy, ARC-Challenge, and OpenBookQA. Under matched optimization, target-module, and evaluation settings, ChainDoRA achieves a seven-task average accuracy of 72.30%, compared with 69.88% for LoRA and 69.39% for DoRA. We further investigate the effects of TT rank and adapter placement, demonstrating a tunable parameter–accuracy trade-off. In addition, we analyze ChainDoRA's parameter scaling across multiple LLM architectures and model sizes to characterize how the proposed factorization scales beyond LLaMA-7B.

The main contributions of this work are summarized as follows:

- We introduce *ChainDoRA*, a magnitude–direction PEFT framework that replaces the densely parameterized directional factors of DoRA with factors obtained from a connected Tensor-Train representation.
- We formulate the adapter rank r as the boundary rank connecting the input- and output-side TT contractions while introducing an independent internal TT rank ρ. This separates the nominal low-rank adaptation dimension from the parameter capacity of the tensor representation.
- We develop a parameter-efficient implementation in which the TT-derived directional update is integrated with row-wise magnitude normalization, together with a balanced nonzero TT initialization designed for stable optimization of the multiplicative tensor chain.
- We conduct a controlled comparison with LoRA and DoRA on seven commonsense reasoning benchmarks using LLaMA-7B. ChainDoRA achieves a 72.30% average accuracy with 5.35M trainable parameters, improving over both baselines while reducing the trainable parameter count by approximately an order of magnitude.
- We provide systematic ablations of TT rank and adaptation location, together with a cross-architecture parameter-scaling analysis, to characterize the efficiency–capacity trade-offs of the proposed parameterization.

The remainder of this paper is organized as follows. Section 2 reviews low-rank, weight-decomposed, and tensor-based PEFT methods. Section 3 presents the ChainDoRA formulation, TT construction, initialization strategy, and parameter complexity. Section 4 describes the experimental protocol. The empirical results and ablation studies are then presented and discussed, followed by the conclusion.

2 Related Work

2.1 Parameter-Efficient and Low-Rank Adaptation

Parameter-efficient fine-tuning (PEFT) adapts pretrained models while updating only a small subset of parameters, thereby reducing the memory and storage requirements associated with conventional full-parameter fine-tuning. Early PEFT approaches introduced lightweight trainable modules between frozen Transformer layers [1], while prefix tuning optimized continuous task-specific vectors that are prepended to the hidden states of Transformer layers [2]. These approaches established that effective downstream adaptation does not necessarily require updating the entire pretrained parameter set.

Low-Rank Adaptation (LoRA) [3] instead reparameterizes the update of a frozen weight matrix W_0 as the product of two trainable low-rank matrices. For a rank-r adapter, the update takes the form BA, where $A \in \mathbb{R}^{r \times d_{\mathrm{in}}}$

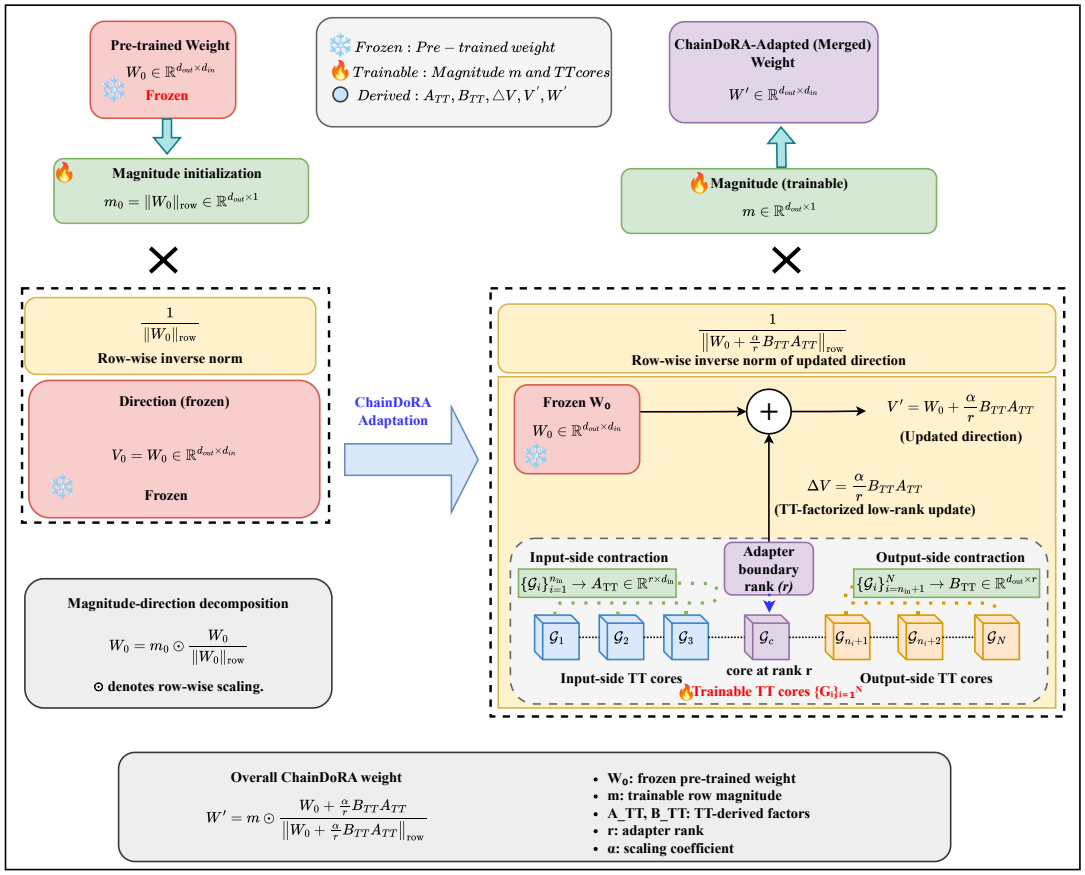


Figure 1: Overview of the proposed ChainDoRA framework. A connected Tensor-Train chain constructs the directional factors A_{TT} and B_{TT}, which are coupled through the adapter-rank boundary r. Their product defines the low-rank directional update, while the row-wise magnitude vector m is optimized independently. The pretrained weight W_0 remains frozen.

and $B \in \mathbb{R}^{d_{\text{out}} \times r}$. This formulation substantially reduces the number of trainable parameters relative to full fine-tuning and allows the learned update to be merged into the pretrained weight for inference. LoRA has subsequently served as the basis for numerous PEFT variants that modify rank allocation, parameter sharing, initialization, or backbone precision.

AdaLoRA [9], for example, parameterizes incremental updates using a singular-value-decomposition-inspired formulation and dynamically reallocates the available rank budget according to parameter importance. QLoRA [10] addresses a complementary source of memory cost by backpropagating through a frozen 4-bit quantized backbone while optimizing LoRA adapters. VeRA [11] further reduces task-specific storage by sharing a pair of frozen random low-rank matrices across layers and learning comparatively small scaling vectors. These methods demonstrate that parameter efficiency can be improved through rank allocation, quantization, and parameter sharing; nevertheless, conventional LoRA and many of its derivatives retain an explicitly matrix-factorized representation of the task-specific update.

2.2 Magnitude–Direction Adaptation

Weight-Decomposed Low-Rank Adaptation (DoRA) [4] analyzes fine-tuning through the decomposition of a pretrained weight into magnitude and directional components. Instead of applying a low-rank update directly to the complete weight matrix, DoRA learns the magnitude independently and uses LoRA to adapt the directional component. The resulting weight can be written conceptually as a learned magnitude multiplied by a normalized, low-rank-updated direction. This decomposition was motivated by observed differences between the magnitude and directional changes produced by full fine-tuning and conventional LoRA. DoRA was shown to improve over LoRA across language, vision-language, and multimodal adaptation tasks while preserving the ability to merge the adapted weights for inference [4].

Despite this decomposition, the directional branch of DoRA is still parameterized through the two dense rank-r factors inherited from LoRA. For an adapted matrix with dimensions $d_{\text{out}} \times d_{\text{in}}$, these factors require $r(d_{\text{in}} + d_{\text{out}})$ trainable parameters before accounting for the magni-

tude vector. Consequently, the directional adapter size continues to grow linearly with the dimensions of the target transformation. ChainDoRA retains the magnitude–direction decomposition of DoRA but replaces these directly optimized directional matrices with factors derived from a structured Tensor-Train representation.

2.3 Tensorized Parameter-Efficient Fine-Tuning

Tensor decomposition provides a structured alternative to conventional matrix factorization for representing high-dimensional parameters. The Tensor-Train (TT) format introduced by Oseledets [5] represents a high-order tensor through a sequence of low-order cores connected by auxiliary ranks. The number of parameters is determined by the physical mode sizes and TT ranks rather than by explicitly storing the complete tensor. TT representations were subsequently applied to neural networks by tensorizing dense weight matrices and replacing them with compact TT structures [6].

Recent work has extended tensor-network representations to PEFT. LoRETTA [7] proposes two TT-based adaptation strategies. $\text{LoRETTA}_{\text{adp}}$ tensorizes lightweight adapter modules, whereas $\text{LoRETTA}_{\text{rep}}$ performs weight reparameterization using compact TT factors. The latter demonstrates that Tensor-Train structure can provide substantial parameter reductions during LLM fine-tuning. TT-LoRA [8] further investigates Tensor-Train parameterization for LLM adaptation and studies the trade-off among tensorization structure, TT rank, parameter count, and downstream performance. In contrast to standard LoRA, these approaches use higher-order tensor structure to reduce the number of explicitly optimized adaptation parameters.

Other tensorized methods adopt different decompositions or initialization strategies. DoTA [12] introduces Weight-Decomposed Tensor Adaptation based on a Matrix Product Operator (MPO) representation and leverages a decomposition of pretrained weights to obtain an informative initialization for tensor adaptation. Its quantized extension, QDoTA, combines this formulation with low-bit backbone quantization. Although DoTA and ChainDoRA both involve weight-decomposed adaptation, their parameterizations are fundamentally different: DoTA adapts weights through an MPO-based tensor representation initialized from pretrained weights, whereas ChainDoRA retains the DoRA magnitude–direction formulation and tensorizes the low-rank directional factors.

TensorGuide [13] is particularly related to the present work because it considers the interaction between the two low-rank matrices rather than treating them as independent objects. TensorGuide employs a unified TT network driven by controlled Gaussian inputs to jointly generate correlated low-rank adaptation matrices. This contrasts with separately tensorizing the two LoRA factors and demonstrates that coupling their parameterization can improve the efficiency and expressivity of tensorized low-rank adaptation.

ChainDoRA shares the general motivation of exploiting dependencies between low-rank factors but differs in both construction and placement of the tensor representation. Specifically, ChainDoRA forms a single connected sequence of trainable TT cores whose input-side and output-side contractions directly produce $A_{\text{TT}} \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B_{\text{TT}} \in \mathbb{R}^{d_{\text{out}} \times r}$. The TT bond separating the two contractions is explicitly assigned the adapter rank r, while the remaining bonds are governed by an independent TT rank ρ. The resulting product $B_{\text{TT}} A_{\text{TT}}$ is then used specifically to modify the directional component of a DoRA-style magnitude–direction decomposition. Thus, the adapter rank controlling the effective low-rank update and the TT rank controlling the internal tensor capacity are explicitly separated in the proposed formulation.

Table 1 summarizes these distinctions. Rather than treating tensor decomposition as an independent adapter or as a replacement for the complete adapted weight, ChainDoRA uses a connected TT representation specifically within the directional branch of magnitude–direction adaptation. This construction motivates the formulation developed in Section 3.

3 Methodology

The proposed ChainDoRA framework combines the magnitude–direction reparameterization of DoRA [4] with a connected Tensor-Train (TT) representation [5] of the directional low-rank factors. Instead of directly optimizing the two dense low-rank matrices used by LoRA and DoRA, ChainDoRA constructs these factors from contractions of a single TT chain. The adapter rank r is assigned to the TT bond separating the input-side and output-side contractions, while a distinct internal TT rank ρ controls the capacity and parameter cost of the tensorized representation. The pretrained weight remains frozen, and only the TT cores and the row-wise magnitude parameters are optimized.

Fig. 1 provides the overall architecture, while Fig. 2 illustrates the internal TT construction used to generate the two directional factors.

3.1 Preliminaries: LoRA and Weight-Decomposed Adaptation

Consider a pretrained linear transformation with frozen weight

$$W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}, \tag{1}$$

where d_{in} and d_{out} denote the input and output dimensions, respectively. LoRA [3] represents a task-specific weight update as

Table 1: Conceptual comparison of representative low-rank and tensorized parameter-efficient adaptation methods. "Joint A/B structure" indicates whether the two low-rank factors are generated or parameterized through a shared structured representation.

Method	Low-Rank Update	Tensor Representation	Magnitude–Direction	Joint A/B Structure	Primary Structural Mechanism
LoRA [3]	✓	–	–	–	Dense trainable low-rank factors
AdaLoRA [9]	✓	–	–	–	Adaptive rank-budget allocation
DoRA [4]	✓	–	✓	–	Magnitude–direction decomposition
LoRETTA [7]	✓	TT	–	–	Tensorized adapters / reparameterization
TT-LoRA [8]	✓	TT	–	–	TT-parameterized adaptation
DoTA [12]	–	MPO	Weight-decomposed	–	Pretrained-weight MPO initialization
TensorGuide [13]	✓	TT	–	✓	Unified TT-generated correlated factors
ChainDoRA	✓	**TT**	✓	✓	**Connected TT directional factors with boundary rank r**

$$\Delta W_{\mathrm{LoRA}} = sBA, \qquad s = \frac{\alpha}{r}, \tag{2}$$

where

$$A \in \mathbb{R}^{r \times d_{\mathrm{in}}}, \qquad B \in \mathbb{R}^{d_{\mathrm{out}} \times r}. \tag{3}$$

Here, r is the low-rank adaptation dimension and α is the LoRA scaling coefficient. Since

$$\mathrm{rank}(BA) \le r, \tag{4}$$

the update matrix has rank at most r, reducing the number of trainable degrees of freedom relative to a dense update while the pretrained matrix W_0 remains unchanged.

DoRA [4] further decomposes adaptation into magnitude and direction. For the $d_{\mathrm{out}} \times d_{\mathrm{in}}$ weight-storage convention used in our implementation, the decomposition is performed row-wise. The initial magnitude is therefore defined as

$$m_i^{(0)} = \|W_{0,i:}\|_2, \qquad i = 1, \dots, d_{\mathrm{out}}, \tag{5}$$

or equivalently,

$$m^{(0)} = \|W_0\|_{\mathrm{row}}. \tag{6}$$

Given an unnormalized directional matrix V, the corresponding magnitude–direction reconstruction is

$$W = m \odot \frac{V}{\|V\|_{\mathrm{row}}}, \tag{7}$$

where $\odot$ denotes row-wise broadcasting. In conventional DoRA, the directional matrix is updated using the LoRA factors,

$$V_{\mathrm{DoRA}} = W_0 + sBA. \tag{8}$$

ChainDoRA preserves the decomposition in (7) but replaces the directly parameterized matrices A and B with factors generated from a connected Tensor-Train representation.

3.2 Connected Tensor-Train Directional Factorization

Let the input and output dimensions be factorized into smaller physical modes,

$$d_{\mathrm{in}} = \prod_{j=1}^{p} n_j^{\mathrm{in}}, \qquad d_{\mathrm{out}} = \prod_{j=1}^{q} n_j^{\mathrm{out}}, \tag{9}$$

where p and q denote the numbers of input-side and output-side modes, respectively. These modes are concatenated into a single sequence

$$(n_1, \dots, n_N) = (n_1^{\mathrm{in}}, \dots, n_p^{\mathrm{in}}, n_1^{\mathrm{out}}, \dots, n_q^{\mathrm{out}}), \qquad N = p+q. \tag{10}$$

Following the TT representation [5], ChainDoRA introduces N trainable third-order cores

$$\mathcal{G}_j \in \mathbb{R}^{R_{j-1} \times n_j \times R_j}, \qquad j = 1, \dots, N, \tag{11}$$

with boundary ranks

$$R_0 = R_N = 1. \tag{12}$$

A central design choice of ChainDoRA is to assign the TT bond between the input-side and output-side contractions directly to the adapter rank,

$$R_p = r. \tag{13}$$

The remaining non-boundary TT ranks are controlled by a separate internal rank ρ,

$$R_j = \begin{cases} 1, & j = 0 \text{ or } j = N, \\ r, & j = p, \\ \rho, & \text{otherwise.} \end{cases} \tag{14}$$

Consequently, the complete rank sequence can be written compactly as

$$\mathbf{R} = \left(1, \underbrace{\rho, \dots, \rho}_{p-1}, r, \underbrace{\rho, \dots, \rho}_{q-1}, 1\right). \tag{15}$$

The role of r is therefore distinct from that of ρ: r controls the maximum rank of the resulting directional update, whereas ρ controls the capacity of the TT representation used to construct its factors.

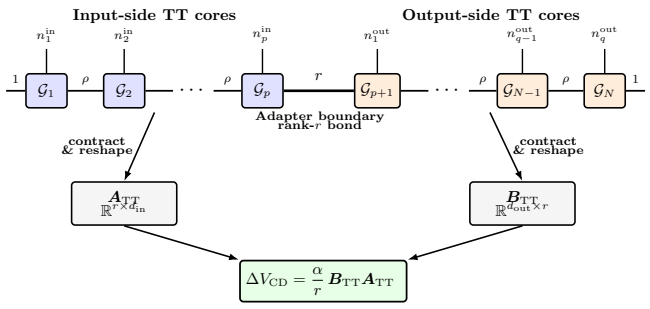


Figure 2: Connected Tensor-Train construction of ChainDoRA. The input-side and output-side TT contractions generate A_{TT} and B_{TT}, respectively. The adapter rank r forms the boundary bond between the two contractions, while the remaining internal TT bonds are controlled by rank ρ.

For a fixed physical index i_j, define the corresponding TT-core matrix slice as

$$G_j^{(i_j)} = \mathcal{G}_j(:, i_j, :) \in \mathbb{R}^{R_{j-1} \times R_j}. \tag{16}$$

Let $\phi_{\mathrm{in}}(\cdot)$ and $\phi_{\mathrm{out}}(\cdot)$ denote the flattening maps from the tensorized input and output indices to the original matrix coordinates. The contraction of the first p cores yields a $1 \times r$ vector for each input coordinate. The corresponding column of A_{TT} is

$$A_{\mathrm{TT}}\left[:, \phi_{\mathrm{in}}(i_1, \ldots, i_p)\right] = \left(G_1^{(i_1)} G_2^{(i_2)} \cdots G_p^{(i_p)}\right)^{\mathrm{T}}. \tag{17}$$

Similarly, contraction of the output-side cores yields an $r \times 1$ vector, and the corresponding row of B_{TT} is

$$B_{\mathrm{TT}}\left[\phi_{\mathrm{out}}(o_1, \ldots, o_q), :\right] = \left(G_{p+1}^{(o_1)} \cdots G_N^{(o_q)}\right)^{\mathrm{T}}. \tag{18}$$

Therefore,

$$A_{\mathrm{TT}} \in \mathbb{R}^{r \times d_{\mathrm{in}}}, \qquad B_{\mathrm{TT}} \in \mathbb{R}^{d_{\mathrm{out}} \times r}. \tag{19}$$

Importantly, A_{TT} and B_{TT} are not stored as independent trainable dense matrices. Their entries are differentiable functions of the TT cores on the two sides of the shared rank-r boundary.

The ChainDoRA directional residual is defined as

$$\Delta V_{\mathrm{CD}} = s B_{\mathrm{TT}} A_{\mathrm{TT}}, \qquad s = \frac{\alpha}{r}. \tag{20}$$

Since the intermediate dimension is still r,

$$\mathrm{rank}\left(\Delta V_{\mathrm{CD}}\right) \le r, \tag{21}$$

regardless of the internal TT rank ρ. Increasing ρ therefore increases the representational capacity of the TT factors without changing the nominal upper bound on the rank of the final directional update.

3.3 Magnitude–Direction Composition

The unnormalized ChainDoRA directional matrix is

$$V_{\mathrm{CD}} = W_0 + s B_{\mathrm{TT}} A_{\mathrm{TT}}. \tag{22}$$

The effective adapted weight is obtained through row-wise normalization and magnitude scaling,

$$W_{\mathrm{CD}} = m \odot \frac{V_{\mathrm{CD}}}{\|V_{\mathrm{CD}}\|_{\mathrm{row}}}. \tag{23}$$

Substituting (22) gives the complete ChainDoRA reparameterization,

$$W_{\mathrm{CD}} = m \odot \frac{W_0 + s B_{\mathrm{TT}} A_{\mathrm{TT}}}{\|W_0 + s B_{\mathrm{TT}} A_{\mathrm{TT}}\|_{\mathrm{row}}}. \tag{24}$$

The set of trainable parameters of an adapted layer is therefore

$$\Theta_{\mathrm{CD}} = \{m, \mathcal{G}_1, \ldots, \mathcal{G}_N\}, \tag{25}$$

while W_0 remains frozen throughout fine-tuning.

3.4 Factored Row-Norm Computation

Direct evaluation of $\|W_0 + s B_{\mathrm{TT}} A_{\mathrm{TT}}\|_{\mathrm{row}}$ by first constructing the dense $d_{\mathrm{out}} \times d_{\mathrm{in}}$ matrix $B_{\mathrm{TT}} A_{\mathrm{TT}}$ introduces unnecessary temporary storage. The required norm can instead be evaluated through low-rank intermediate quantities. A related factored-norm formulation has recently been studied for scaling DoRA implementations [14].

For notational simplicity, let

$$A = A_{\mathrm{TT}}, \qquad B = B_{\mathrm{TT}}. \tag{26}$$

For output row i,

$$\begin{aligned} \|W_{0,i:} + s B_{i:} A\|_2^2 = {} & \|W_{0,i:}\|_2^2 \\ & + 2s \left\langle W_{0,i:}, B_{i:} A \right\rangle \\ & + s^2 \left\|B_{i:} A\right\|_2^2. \end{aligned} \tag{27}$$

Define

$$q_i = \|W_{0,i:}\|_2^2, \tag{28}$$

$$c_i = \sum_{k=1}^{r} B_{ik} \left(W_0 A^{\mathrm{T}}\right)_{ik}, \tag{29}$$

and

$$u_i = B_{i:} \left(A A^{\mathrm{T}}\right) B_{i:}^{\mathrm{T}}. \tag{30}$$

The row norm is then

$$n_i = \sqrt{q_i + 2sc_i + s^2 u_i + \epsilon}, \tag{31}$$

where ϵ is a small numerical-stability constant. The dominant rank-dependent intermediate matrices are

$$W_0 A^{\mathrm{T}} \in \mathbb{R}^{d_{\mathrm{out}} \times r} \tag{32}$$

and

$$AA^{\mathrm{T}} \in \mathbb{R}^{r \times r}, \tag{33}$$

rather than the full $d_{\mathrm{out}} \times d_{\mathrm{in}}$ matrix BA.

The linear action of the low-rank residual can likewise be evaluated sequentially. For an input x,

$$z = A_{\mathrm{TT}} x, \qquad z \in \mathbb{R}^r, \tag{34}$$

followed by

$$\Delta y = B_{\mathrm{TT}} z. \tag{35}$$

Let

$$g_i = \frac{m_i}{n_i}. \tag{36}$$

The complete forward transformation can therefore be expressed as

$$y = g \odot \left[W_0 x + s B_{\mathrm{TT}} \left(A_{\mathrm{TT}} x \right) \right]. \tag{37}$$

This formulation avoids materializing $B_{\mathrm{TT}} A_{\mathrm{TT}}$ solely for either the linear residual or the row-norm computation.

3.5 Detached Direction Normalization

The original DoRA formulation notes that retaining the normalization term in the complete gradient graph introduces additional training-memory overhead [4]. Following its low-memory modification, our implementation treats the dynamically computed directional norm as a constant during backpropagation.

Let

$$\bar{n} = \mathrm{stopgrad}\left(\left\| V_{\mathrm{CD}} \right\|_{\mathrm{row}} \right). \tag{38}$$

The training-time reparameterization is therefore

$$W_{\mathrm{CD}} = m \odot \frac{V_{\mathrm{CD}}}{\bar{n}}. \tag{39}$$

The forward numerical value is unchanged by the stop-gradient operation; only gradient propagation through the normalization denominator is removed. We use this detached-denominator configuration for the reported ChainDoRA experiments.

3.6 Balanced Nonzero TT Initialization

Initialization requires additional care for a multiplicative TT parameterization. Conventional LoRA commonly initializes one low-rank factor to zero so that the initial residual vanishes. Directly transferring this strategy to a chain of multiple tensor factors is undesirable because a zero factor can suppress gradients to other multiplicative factors. A similar zero-gradient issue for zero-initialized tensor factors has been reported for TT-based PEFT in LoRETTA [7].

ChainDoRA therefore initializes every TT core with nonzero, fan-in-scaled Gaussian entries. The magnitude vector is initialized from the frozen pretrained weight according to

$$m^{(0)} = \left\| W_0 \right\|_{\mathrm{row}}. \tag{40}$$

Because products of several tensor cores can develop strongly imbalanced scales, we next apply gauge-preserving balancing between neighboring TT cores. For any positive scalar λ_j, the transformation

$$\mathcal{G}_j \leftarrow \lambda_j \mathcal{G}_j, \qquad \mathcal{G}_{j+1} \leftarrow \lambda_j^{-1} \mathcal{G}_{j+1} \tag{41}$$

leaves their joint contraction unchanged. We use this freedom to reduce large scale disparities between adjacent cores.

After contraction, the two directional factors are additionally balanced across the adapter boundary. Let

$$a_{\mathrm{rms}} = \mathrm{RMS}\left(A_{\mathrm{TT}} \right), \qquad b_{\mathrm{rms}} = \mathrm{RMS}\left(B_{\mathrm{TT}} \right). \tag{42}$$

Choosing

$$\gamma = \sqrt{\frac{b_{\mathrm{rms}}}{a_{\mathrm{rms}}}} \tag{43}$$

and applying

$$A_{\mathrm{TT}} \leftarrow \gamma A_{\mathrm{TT}}, \qquad B_{\mathrm{TT}} \leftarrow \gamma^{-1} B_{\mathrm{TT}} \tag{44}$$

preserves the directional product,

$$\left(\gamma^{-1} B_{\mathrm{TT}} \right) \left(\gamma A_{\mathrm{TT}} \right) = B_{\mathrm{TT}} A_{\mathrm{TT}}, \tag{45}$$

while reducing scale imbalance between the two sides of the TT chain.

Unlike exact zero initialization, ChainDoRA starts with a small but nonzero directional perturbation. We explicitly calibrate its relative magnitude as

$$\delta_0 = \frac{\left\| s B_{\mathrm{TT}}^{(0)} A_{\mathrm{TT}}^{(0)} \right\|_{\mathrm{F}}}{\left\| W_0 \right\|_{\mathrm{F}}}. \tag{46}$$

For all experiments, the initialization is rescaled to satisfy

$$\delta_0 = \tau, \qquad \tau = 10^{-3}. \tag{47}$$

Thus, ChainDoRA begins in a controlled neighborhood of the pretrained transformation while keeping all TT factors nonzero and trainable from the first optimization step.

The calibration does not require materializing the dense product $B_{\mathrm{TT}} A_{\mathrm{TT}}$. Its Frobenius norm follows from the $r \times r$ Gram matrices,

Algorithm 1 ChainDoRA Training and Weight Merging

Input: Frozen weights $\{W_0^{(\ell)}\}$, data $\mathcal{D}$, adapter rank r, TT rank ρ, scale α, target τ
$s \leftarrow \alpha/r$
Initialization
for each adapted layer ℓ **do**
  Tensorize $d_{\text{in}}^{(\ell)}$ and $d_{\text{out}}^{(\ell)}$
  Set $\mathbf{R} = (1, \rho, \ldots, \rho, r, \rho, \ldots, \rho, 1)$
  Initialize all $\{\mathcal{G}_j^{(\ell)}\}_{j=1}^N$ with nonzero fan-in-scaled Gaussian entries
  $m^{(\ell)} \leftarrow \|W_0^{(\ell)}\|_{\text{row}}$
  Gauge-balance neighboring TT cores and contract them into $A_{\text{TT}}^{(\ell)}$ and $B_{\text{TT}}^{(\ell)}$
  Balance the two factors across the rank-r boundary
  Calibrate $\delta_0^{(\ell)} = \|sB_{\text{TT}}^{(\ell)}A_{\text{TT}}^{(\ell)}\|_F / \|W_0^{(\ell)}\|_F \approx \tau$
end for
Fine-tuning
for each mini-batch $(x, y) \in \mathcal{D}$ **do**
  for each adapted layer ℓ **do**
    Contract TT cores into $A_{\text{TT}}^{(\ell)}$ and $B_{\text{TT}}^{(\ell)}$
    $\Delta y^{(\ell)} \leftarrow sB_{\text{TT}}^{(\ell)}(A_{\text{TT}}^{(\ell)}x)$
    Compute factored row norm $n^{(\ell)}$ using Eqs. (28)–(31)
    $\bar{n}^{(\ell)} \leftarrow \text{stopgrad}(n^{(\ell)})$
    $y^{(\ell)} \leftarrow (m^{(\ell)} \oslash \bar{n}^{(\ell)}) \odot [W_0^{(\ell)}x + \Delta y^{(\ell)}]$
  end for
  Compute $\mathcal{L}_{\text{resp}}$ and update only $\{m^{(\ell)}, \mathcal{G}_{1:N}^{(\ell)}\}$ with AdamW
end for
Weight merging
for each adapted layer ℓ **do**
  Contract final TT cores into $A_{\text{TT}}^{(\ell)}$ and $B_{\text{TT}}^{(\ell)}$
  $V_{\text{CD}}^{(\ell)} \leftarrow W_0^{(\ell)} + sB_{\text{TT}}^{(\ell)}A_{\text{TT}}^{(\ell)}$
  $W_{\text{CD}}^{(\ell)} \leftarrow m^{(\ell)} \odot V_{\text{CD}}^{(\ell)} / \|V_{\text{CD}}^{(\ell)}\|_{\text{row}}$
end for
Return: $\{W_{\text{CD}}^{(\ell)}\}$

$$\|B_{\text{TT}}A_{\text{TT}}\|_{\text{F}}^2 = \text{tr}\left[\left(B_{\text{TT}}^{\text{T}}B_{\text{TT}}\right)\left(A_{\text{TT}}A_{\text{TT}}^{\text{T}}\right)\right]. \tag{48}$$

This allows initialization calibration to be performed using low-dimensional intermediate quantities.

Algorithm 1 summarizes the complete ChainDoRA training pipeline, including TT-core initialization, construction of the directional factors, factored row normalization, parameter optimization, and final weight merging.

3.7 Trainable-Parameter Complexity

For a rank-r LoRA adapter, the number of trainable parameters associated with one adapted matrix is

$$P_{\text{LoRA}} = r\,(d_{\text{in}} + d_{\text{out}})\,. \tag{49}$$

DoRA additionally learns a magnitude parameter for each output row, giving

$$P_{\text{DoRA}} = r\,(d_{\text{in}} + d_{\text{out}}) + d_{\text{out}}. \tag{50}$$

For ChainDoRA, the dense low-rank matrices are replaced by the TT cores. The number of trainable TT parameters is the standard TT storage cost

$$P_{\text{TT}} = \sum_{j=1}^{N} R_{j-1}n_jR_j. \tag{51}$$

Including the trainable magnitude vector,

$$P_{\text{ChainDoRA}} = d_{\text{out}} + \sum_{j=1}^{N} R_{j-1}n_jR_j. \tag{52}$$

The relative parameter ratio with respect to DoRA is therefore

$$\eta_{\text{CD/DoRA}} = \frac{d_{\text{out}} + \sum_{j=1}^{N} R_{j-1}n_jR_j}{r(d_{\text{in}} + d_{\text{out}}) + d_{\text{out}}}. \tag{53}$$

Correspondingly, the percentage reduction is

$$\mathcal{R}_{\text{CD/DoRA}} = 100\left(1 - \eta_{\text{CD/DoRA}}\right). \tag{54}$$

Equation (52) also makes the role of the internal TT rank explicit: reducing ρ decreases the TT-core parameter count without changing the adapter boundary rank r. This provides a direct mechanism for controlling the parameter–capacity trade-off, which is examined empirically in the TT-rank ablation study.

3.8 Adapter Merging and Inference

After fine-tuning, the TT cores are contracted to obtain A_{TT} and B_{TT}, and the final effective weight is computed using (24). The resulting matrix W_{CD} can then replace the corresponding frozen pretrained weight. Consequently, the TT adapter does not require a separate inference branch after merging, analogous to the mergeable property of LoRA and DoRA [3, 4].

The proposed factorization is primarily intended to reduce the number of trainable and stored task-specific parameters. This reduction should not be interpreted as an equivalent reduction in training FLOPs, because contracting TT cores and evaluating the normalized directional update introduce additional operations. We therefore report parameter efficiency and predictive performance separately in the experimental analysis.

4 Experimental Setup

We evaluate ChainDoRA under a controlled parameter-efficient adaptation setting designed to isolate the effect of the proposed TT-factorized directional parameterization. All primary experiments use the same pretrained

backbone, adaptation rank, target modules, optimization schedule, training data, and evaluation pipeline for LoRA, DoRA, and ChainDoRA. Unless otherwise stated, only the adapter parameterization differs between methods.

4.1 Backbone and Training Data

We use LLaMA-7B [15] as the frozen pretrained backbone. LLaMA is a decoder-only Transformer family containing models ranging from 7B to 65B parameters [15]. The 7B configuration is used throughout the primary experiments to maintain a fixed backbone while comparing the parameter efficiency of different adaptation methods.

Our implementation follows the commonsense-reasoning adaptation pipeline used in the public DoRA framework [4]. Rather than training on the complete commonsense instruction collection, we construct a fixed adaptation pool containing 15,119 instruction–response examples. The adaptation pool is derived from the Commonsense170K collection introduced with LLM-Adapters [16]. A validation subset of 120 examples is held out using a fixed random seed of 42, leaving 14,999 examples for parameter optimization. All LoRA, DoRA, ChainDoRA, and ChainDoRA ablation experiments use exactly the same split.

Each training instance consists of an instruction and its target response. We employ *response-only supervision*: tokens belonging to the instruction portion of the prompt are masked from the language-modeling loss, and only response tokens contribute to optimization. Let $\mathbf{y} = (y_1, \ldots, y_T)$ denote the tokenized instruction–response sequence, and let $\mathcal{S}$ denote the token positions corresponding to the target response. The training objective is

$$\mathcal{L}_{\text{resp}} = -\frac{1}{|\mathcal{S}|} \sum_{t \in \mathcal{S}} \log p_\theta \left(y_t \mid y_{<t} \right), \tag{55}$$

where tokens outside $\mathcal{S}$ are ignored when computing the loss. The same serialized instruction template and response delimiter are used during fine-tuning and evaluation to avoid train–test differences caused by prompt formatting.

4.2 Adaptation Configuration

For the main comparison, LoRA, DoRA, and ChainDoRA use the same adapter rank

$$r = 32 \tag{56}$$

and scaling coefficient

$$\alpha = 64. \tag{57}$$

Adaptation is applied to five linear transformations in every Transformer block,

Table 2: Tensorization used for ChainDoRA on LLaMA-7B.

Module	d_{in}	d_{out}	Input Modes	Output Modes
q_{proj}	4096	4096	$(16, 16, 16)$	$(16, 16, 16)$
k_{proj}	4096	4096	$(16, 16, 16)$	$(16, 16, 16)$
v_{proj}	4096	4096	$(16, 16, 16)$	$(16, 16, 16)$
up_{proj}	4096	11008	$(16, 16, 16)$	$(16, 16, 43)$
$down_{\text{proj}}$	11008	4096	$(16, 16, 43)$	$(16, 16, 16)$

$$\mathcal{M}_{\text{full}} = \{q_{\text{proj}}, k_{\text{proj}}, v_{\text{proj}}, up_{\text{proj}}, down_{\text{proj}}\}. \tag{58}$$

For the 32 Transformer layers of LLaMA-7B, this corresponds to 160 adapted linear modules.

The main ChainDoRA configuration uses internal TT rank

$$\rho = 16. \tag{59}$$

The hidden dimension of LLaMA-7B is tensorized as

$$4096 = 16 \times 16 \times 16, \tag{60}$$

whereas the feed-forward dimension is represented as

$$11008 = 16 \times 16 \times 43. \tag{61}$$

Table 2 summarizes the resulting mode assignments.

Under this configuration, conventional LoRA contains 56.10M trainable parameters and DoRA contains 56.98M trainable parameters. ChainDoRA with $\rho = 16$ requires 5.35M trainable parameters while preserving the same adapter boundary rank $r = 32$. Exact parameter-efficiency comparisons are reported in Section 5.

4.3 Optimization and Implementation Details

All methods are fine-tuned for three epochs using AdamW. The learning rate is set to

$$\eta = 2 \times 10^{-4}, \tag{62}$$

with zero weight decay and 100 warm-up steps. The global batch size and micro-batch size are both 4, resulting in one optimizer update per micro-batch on the single-GPU setup. The maximum tokenized sequence length is 256 and the adapter dropout probability is 0.05.

Training uses FP16 precision and gradient checkpointing. Validation is performed every 80 optimization steps, and adapter checkpoints are saved at the same interval. At the end of training, the best validation checkpoint is restored. All experiments are implemented in PyTorch using the PEFT-compatible DoRA codebase extended with the proposed ChainDoRA module. Training is conducted on a single NVIDIA GeForce RTX 5090 GPU with 32 GB of memory.

Table 3 summarizes the common training configuration.

Table 3: Training configuration used for the primary comparison.

Configuration	Value
Backbone	LLaMA-7B
Adaptation pool	15,119 examples
Optimization examples	14,999
Validation examples	120
Training epochs	3
Optimizer	AdamW
Learning rate	2×10^{-4}
Weight decay	0
Warm-up steps	100
Global batch size	4
Micro-batch size	4
Maximum sequence length	256
Adapter dropout	0.05
Adapter rank r	32
Scaling coefficient α	64
Main ChainDoRA TT rank ρ	16
Target modules	Q, K, V, Up, Down
Validation interval	80 steps
Checkpoint interval	80 steps
Precision	FP16
Gradient checkpointing	Enabled
Supervision	Response only
Hardware	NVIDIA RTX 5090 (32 GB)

4.4 Commonsense Reasoning Benchmarks

We evaluate the adapted models on seven established commonsense reasoning benchmarks covering complementary forms of linguistic, physical, social, coreference, and science-oriented reasoning.

BoolQ [17] consists of naturally occurring yes/no questions and evaluates binary question answering. PIQA [18] measures physical commonsense reasoning through two candidate solutions to everyday physical situations. SocialIQA [19] evaluates reasoning about social interactions, intentions, and consequences. WinoGrande [20] is a large-scale adversarially constructed Winograd-style benchmark designed to reduce exploitable annotation biases.

ARC [21] consists of natural grade-school science questions and is divided into ARC-Easy and ARC-Challenge. The Challenge partition was constructed from questions that were incorrectly answered by baseline retrieval and co-occurrence systems, making it a more difficult subset. OpenBookQA [22] evaluates elementary science question answering that may require combining a provided scientific fact with broader common knowledge.

Table 4 lists the seven evaluation sets and the number of examples used in our experiments.

4.5 Evaluation Protocol and Metrics

A single evaluation pipeline is used for all methods. Each test example is converted to an instruction containing the question, candidate answers, and the required benchmark-specific output identifier. The same instruction serialization used during fine-tuning is retained during inference, with the response field left empty for generation.

Table 4: Commonsense reasoning benchmarks used for evaluation.

Benchmark	Test Examples	Primary Reasoning Type
BoolQ	3,270	Yes/no reasoning
PIQA	1,838	Physical commonsense
SocialIQA	1,954	Social commonsense
WinoGrande	1,267	Coreference / commonsense
ARC-Easy	2,376	Science reasoning
ARC-Challenge	1,172	Challenging science reasoning
OpenBookQA	500	Science + common knowledge
Total	**12,377**	–

Before evaluation, the learned adapter parameters are merged into their corresponding linear transformations. Generation is deterministic and uses beam search with four beams,

$$N_{\text{beam}} = 4, \tag{63}$$

with a maximum of 32 newly generated tokens,

$$T_{\text{gen}}^{\max} = 32. \tag{64}$$

Sampling is disabled. The generated continuation is mapped to the benchmark-specific answer vocabulary. Examples include `true/false` for BoolQ, `solution1/solution2` for PIQA, `option1/option2` for WinoGrande, and `answer1`, `answer2`, ... for the multiple-choice question-answering datasets. The first valid answer identifier appearing in the generated response is used as the prediction. A generation containing no valid identifier is counted as incorrect.

The primary metric for benchmark j is classification accuracy,

$$\text{Acc}_j = \frac{N_{\text{correct},j}}{N_{\text{total},j}} \times 100. \tag{65}$$

To summarize performance across the complete benchmark suite, we report the unweighted macro-average over the seven tasks,

$$\text{Avg.} = \frac{1}{7} \sum_{j=1}^{7} \text{Acc}_j. \tag{66}$$

The macro-average gives equal weight to each benchmark regardless of the number of test examples.

4.6 Ablation Study Design

We conduct two principal ablation studies to examine the effects of TT capacity and adapter placement.

First, the internal TT rank is varied while keeping the adapter rank fixed at $r = 32$:

Table 5: ChainDoRA configurations used in the ablation studies.

Configuration	ρ	Target Set	Trainable Params.
TT16-Full	16	QKV+MLP	5,346,816
TT8-Full	8	QKV+MLP	2,784,000
TT4-Full	4	QKV+MLP	1,748,352
TT16-QV	16	QV	1,867,776
TT16-QKV	16	QKV	2,801,664
TT16-MLP	16	MLP	2,545,152
TT16-QV+MLP	16	QV+MLP	4,412,928
TT8-MLP	8	MLP	1,383,168

$$\rho \in \{4, 8, 16\}. \tag{67}$$

All five projection types in $\mathcal{M}_{\text{full}}$ remain adapted in this experiment. This ablation isolates the effect of TT representation capacity without changing the nominal rank of the directional update.

Second, we vary the locations at which ChainDoRA is inserted. The evaluated target sets are

$$\mathcal{M}_{\text{QV}} = \{q_{\text{proj}}, v_{\text{proj}}\}, \tag{68}$$

$$\mathcal{M}_{\text{QKV}} = \{q_{\text{proj}}, k_{\text{proj}}, v_{\text{proj}}\}, \tag{69}$$

$$\mathcal{M}_{\text{MLP}} = \{up_{\text{proj}}, down_{\text{proj}}\}, \tag{70}$$

and

$$\mathcal{M}_{\text{QV+MLP}} = \{q_{\text{proj}}, v_{\text{proj}}, up_{\text{proj}}, down_{\text{proj}}\}. \tag{71}$$

These configurations are compared with full QKV+MLP adaptation. Unless otherwise specified, the projection-location study uses $\rho = 16$.

We additionally evaluate a highly compressed configuration combining $\rho = 8$ with MLP-only adaptation. This variant is included to examine whether reductions in TT capacity and adaptation coverage can be combined while retaining useful downstream performance.

4.7 Cross-Architecture Parameter-Scaling Analysis

To evaluate how the parameterization scales beyond LLaMA-7B, we additionally perform a configuration-level parameter analysis across model families and sizes. This analysis does not require loading pretrained model weights or performing additional fine-tuning. Instead, architectural configurations are used to determine the dimensions and number of target linear transformations, after which the trainable parameter counts of LoRA, DoRA, and ChainDoRA are computed analytically.

The analysis includes Qwen2.5-3B and Qwen2.5-72B [23], Mistral-7B [24], Phi-3-Medium [25], OPT-30B [26], and Falcon-40B [27]. These architectures span approximately 3B to 72B parameters and represent multiple Transformer families with different attention and feed-forward configurations. For each architecture, target modules are selected according to their native attention and feed-forward projection structure. Full-target ChainDoRA is used for the primary apples-to-apples comparison with LoRA and DoRA, while selective MLP-only variants are reported separately as additional compression configurations.

This experiment is intended to characterize parameter scaling rather than downstream accuracy across these model families; no pretrained weights are loaded and no performance claims are made for the configuration-only models.

5 Results and Discussion

This section evaluates ChainDoRA from three complementary perspectives: predictive performance, trainable-parameter efficiency, and sensitivity to the design of the tensorized adapter. We first compare ChainDoRA with matched LoRA and DoRA baselines on the seven commonsense reasoning benchmarks. We then study the effect of the internal TT rank and the placement of ChainDoRA across attention and feed-forward projections. Finally, we examine the parameter scaling of the proposed representation across several large-language-model architectures.

5.1 Overall Commonsense Reasoning Performance

Table 6 reports the primary comparison among LoRA, DoRA, and ChainDoRA. All three methods use the same LLaMA-7B backbone, rank $r = 32$, scaling coefficient $\alpha = 64$, target modules, optimization schedule, response-only training objective, and evaluation protocol. ChainDoRA uses the full QKV+MLP target set with internal TT rank $\rho = 16$.

ChainDoRA achieves the highest macro-average accuracy, reaching 72.30% across the seven benchmarks. This corresponds to improvements of 2.42 percentage points over LoRA and 2.91 percentage points over DoRA. Relative to LoRA, ChainDoRA achieves higher accuracy on six of the seven benchmarks, with particularly notable improvements on PIQA, ARC-Easy, and ARC-Challenge. The respective gains are 5.55, 4.29, and 4.70 percentage points. LoRA obtains the highest SocialIQA score, exceeding ChainDoRA by 0.67 percentage points.

Compared with DoRA, ChainDoRA achieves higher accuracy on all seven tasks. The largest difference is observed on ARC-Challenge, where accuracy increases from 60.07% to 67.58%, corresponding to a 7.51-percentage-point improvement. ARC-Easy increases from 76.60% to 82.32%, a gain of 5.72 percentage points. These results

Table 6: Commonsense reasoning performance of LoRA, DoRA, and ChainDoRA under the controlled 15,119-example adaptation setting. Accuracy is reported in percent. The best accuracy in each benchmark column is shown in bold.

Method	Params.	BoolQ	PIQA	SocialIQA	WinoGrande	ARC-E	ARC-C	OBQA	Avg.
LoRA	56.10M	64.83	72.09	**72.47**	65.82	78.03	62.88	73.00	69.88
DoRA	56.98M	65.60	74.54	70.21	65.51	76.60	60.07	73.20	69.39
ChainDoRA	**5.35M**	**66.24**	**77.64**	71.80	**66.30**	**82.32**	**67.58**	**74.20**	**72.30**

Table 7: Trainable-parameter efficiency of the primary methods. The compression factor indicates how many times larger the corresponding baseline adapter is than ChainDoRA.

Method	Trainable Params.	Compression	Reduction
LoRA	56,098,816	10.49×	90.47%
DoRA	56,975,360	10.66×	90.62%
ChainDoRA	5,346,816	1.00×	–

show that, under the controlled adaptation setting considered here, the connected TT parameterization achieves higher predictive performance while substantially reducing the number of trainable parameters.

DoRA obtains a slightly lower average accuracy than LoRA in our experiments (69.39% versus 69.88%). This observation is specific to the matched training and evaluation configuration used in this work. The original DoRA study reports improvements over LoRA across multiple language and multimodal settings [4]; therefore, the present result should not be interpreted as a general comparison of the learning capabilities of LoRA and DoRA. Instead, the matched baselines establish the relative performance of ChainDoRA within the controlled response-only adaptation regime examined in this study.

5.2 Parameter Efficiency

The predictive improvement of ChainDoRA is accompanied by a substantial reduction in trainable parameters. Table 7 summarizes the parameter requirements of the three primary methods.

At identical adapter rank $r = 32$, ChainDoRA reduces the trainable parameter count by 90.47% relative to LoRA and 90.62% relative to DoRA. Equivalently, LoRA and DoRA require approximately 10.49× and 10.66× more trainable parameters, respectively. Importantly, this reduction is achieved without decreasing the nominal rank bound of the directional update: all three methods use $r = 32$, while ChainDoRA controls the complexity of the factor construction independently through the TT rank ρ.

The relationship between trainable parameter count and average benchmark accuracy is visualized in Fig. 3. ChainDoRA-TT16 occupies a favorable region of this parameter–accuracy space, combining the highest average accuracy among the primary methods with a substantially smaller task-specific parameter footprint.

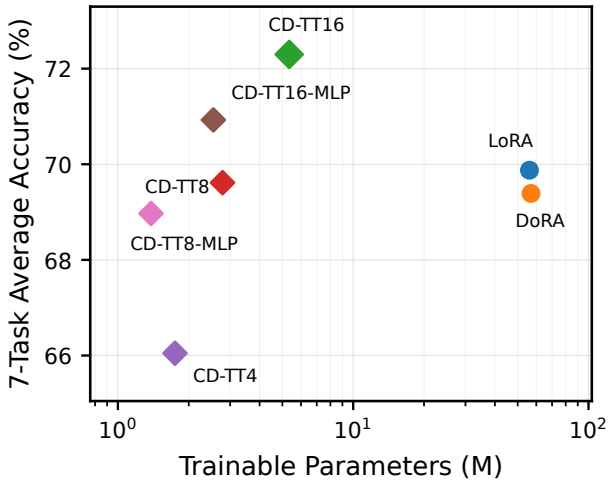


Figure 3: Parameter–accuracy trade-off among LoRA, DoRA, and representative ChainDoRA configurations.

5.3 Effect of Tensor-Train Rank

The internal TT rank ρ determines the capacity of the connected tensor representation without altering the adapter boundary rank r. To evaluate this effect, we vary

$$\rho \in \{4, 8, 16\}, \tag{72}$$

while keeping $r = 32$ and adapting all QKV+MLP projections.

Table 8 shows a monotonic capacity–performance relationship over the evaluated ranks. Increasing ρ from 4 to 8 improves the seven-task average from 66.05% to 69.61%, and increasing ρ to 16 further raises it to 72.30%.

Reducing ρ from 16 to 8 lowers the trainable parameter count from 5.347M to 2.784M, corresponding to a 47.93% reduction, while reducing average accuracy by 2.69 percentage points. At $\rho = 4$, the adapter uses 1.748M trainable parameters, a 67.30% reduction relative to TT16-Full, with a 6.25-percentage-point decrease in average accuracy.

These results empirically support the intended separation between the adapter rank r and the TT capacity parameter ρ. The former fixes the maximum rank of the resulting directional update, whereas the latter provides

Table 8: Effect of the internal TT rank on ChainDoRA under full QKV+MLP adaptation with a fixed adapter rank of $r = 32$. Accuracy is reported in percent. The best accuracy in each benchmark column is shown in bold.

ρ	Params.	BoolQ	PIQA	SocialIQA	WinoGrande	ARC-E	ARC-C	OBQA	Avg.
4	1.748M	57.16	75.63	66.33	59.91	77.90	58.62	66.80	66.05
8	2.784M	64.62	76.61	67.96	63.85	80.98	62.29	71.00	69.61
16	5.347M	**66.24**	**77.64**	**71.80**	**66.30**	**82.32**	**67.58**	**74.20**	**72.30**

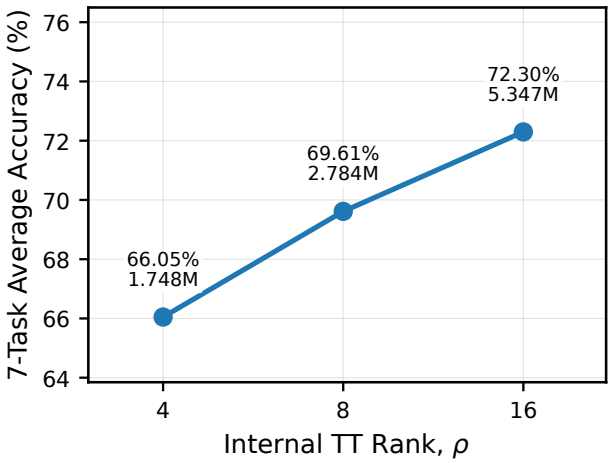


Figure 4: Effect of internal TT rank ρ on the seven-task average accuracy and ChainDoRA parameter count. Increasing ρ improves predictive performance while increasing the size of the TT representation.

an independent mechanism for adjusting the expressiveness and storage cost of its tensorized construction. Similar rank–compression trade-offs have been observed in previous TT-based adaptation studies [7, 8]; however, in ChainDoRA this trade-off occurs specifically within the directional component of magnitude–direction adaptation.

5.4 Effect of Adaptation Location

We next investigate whether the full set of attention and feed-forward projections is necessary for effective ChainDoRA adaptation. Table 9 compares full QKV+MLP adaptation with selective insertion into QV, QKV, MLP, and QV+MLP projections. All variants in this experiment use $\rho = 16$.

Full QKV+MLP adaptation provides the highest overall accuracy. However, selective adaptation reveals that a substantial portion of the performance can be retained with considerably fewer parameters. In particular, MLP-only adaptation achieves 70.93% average accuracy with 2.545M trainable parameters, reducing the adapter size by 52.40% relative to TT16-Full while losing only 1.37 percentage points.

The QV+MLP configuration also achieves 70.93% average accuracy, matching the MLP-only configuration at the reported precision, but requires 4.413M trainable parameters. Thus, adding Q and V projections increases the parameter count by approximately 1.868M without improving the reported macro-average. Among the selective configurations, MLP-only adaptation therefore provides the strongest parameter–accuracy balance.

Table 9: Effect of ChainDoRA adapter placement.

Target	Params. (M)	Avg. (%)	Δ (pp)
QV	1.868	70.43	−1.87
QKV	2.802	68.60	−3.70
MLP	2.545	70.93	−1.37
QV+MLP	4.413	70.93	−1.37
QKV+MLP	5.347	**72.30**	–

Interestingly, QKV adaptation obtains a lower average accuracy than QV adaptation despite using more trainable parameters. This result indicates that increasing the number of adapted projection types does not necessarily produce monotonic performance improvements under a fixed TT configuration. The effectiveness of the adapter therefore depends not only on its total parameter count but also on where the structured update is introduced.

5.5 Combined Rank and Placement Compression

The TT-rank and projection-location experiments suggest two independent ways to reduce ChainDoRA's parameter count: decreasing the internal TT rank and restricting adaptation to a subset of projections. We therefore evaluate a combined configuration using $\rho = 8$ and MLP-only adaptation.

Table 9 compares full QKV+MLP adaptation with selective placement of ChainDoRA. Here, Δ denotes the change in seven-task average accuracy, in percentage points, relative to the full QKV+MLP configuration.

The TT8-MLP variant requires only 1.383M trainable parameters, representing a 74.13% reduction relative to TT16-Full. Compared with TT8-Full, MLP-only placement removes 50.32% of the trainable parameters while reducing average accuracy by only 0.64 percentage points.

Compared with TT4-Full, TT8-MLP uses 20.89% fewer parameters while achieving a 2.92-percentage-point higher

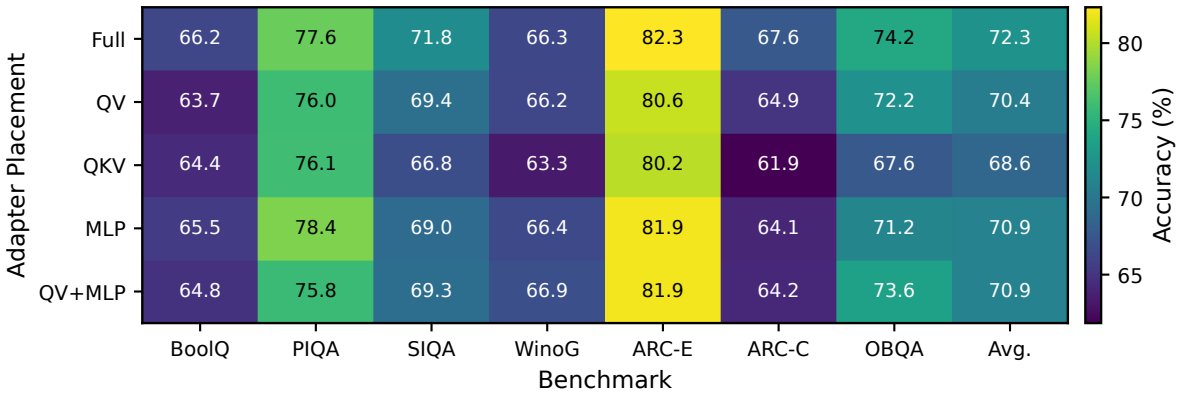


Figure 5: Per-benchmark performance of ChainDoRA under different adapter-placement strategies.

Table 10: Representative ChainDoRA parameter–accuracy trade-offs.

Configuration	Params. (M)	Avg. (%)
TT16-Full	5.347	72.30
TT16-MLP	2.545	70.93
TT8-Full	2.784	69.61
TT8-MLP	1.383	68.97
TT4-Full	1.748	66.05

Table 11: Cross-architecture trainable-parameter scaling.

Model	Params. (M)		Red.	Comp.
	DoRA	CD-TT16	(%)	(×)
Qwen2.5-3B	40.67	4.97	87.77	8.18
Mistral-7B	57.41	5.46	90.48	10.51
Phi-3-Medium	100.25	6.09	93.92	16.45
OPT-30B	178.91	12.42	93.06	14.40
Falcon-40B	193.72	12.12	93.74	15.98
Qwen2.5-72B	286.30	19.75	93.10	14.49

average accuracy. Thus, TT8-MLP strictly dominates TT4-Full with respect to both parameter count and average accuracy within the evaluated configuration space.

5.6 Cross-Architecture Parameter Scaling

The preceding experiments evaluate downstream accuracy on LLaMA-7B. To determine whether the parameter savings are specific to this architecture, we additionally perform the configuration-level analysis described in Section 4.7. This experiment calculates trainable parameter counts from model configurations without loading pretrained weights and therefore measures structural scaling only.

Table 11 compares full-target DoRA with full-target ChainDoRA-TT16 across six representative architectures. Here, CD-TT16 denotes the full-target ChainDoRA configuration with internal TT rank $\rho = 16$.

ChainDoRA-TT16 requires fewer trainable parameters than DoRA for every architecture considered. The compression factor ranges from 8.18× on Qwen2.5-3B to 16.45× on Phi-3-Medium, with an average compression factor of approximately 13.34× across the six configurations. The absolute savings become increasingly large for the larger models; for example, the Qwen2.5-72B configuration requires 286.30M DoRA parameters but only 19.75M ChainDoRA-TT16 parameters under the corresponding target-module mapping. More aggressive selective configurations provide further reductions. For example, the TT8-MLP variant yields compression factors ranging from approximately 28.9× to 43.7× relative to full-target DoRA across the same architecture set. Because these configurations adapt fewer modules, they are treated separately from the primary full-target comparison.

5.7 Discussion and Limitations

The experimental results reveal three main properties of ChainDoRA. First, the connected TT representation can substantially reduce the parameter cost of DoRA-style directional adaptation without requiring a smaller adapter boundary rank. The primary TT16 configuration uses only 5.35M trainable parameters while obtaining the highest average accuracy in the controlled seven-task comparison. This suggests that a densely parameterized pair of directional factors is not necessary to obtain an effective rank-32 update in the studied setting.

Second, parameter efficiency is controlled by more than a single scalar compression parameter. The TT-rank experiment shows that ρ provides a direct capacity–accuracy trade-off, while the projection study demonstrates that adapter placement also has a substantial effect. In particular, MLP-only adaptation retains much of the performance of full adaptation with approximately half of

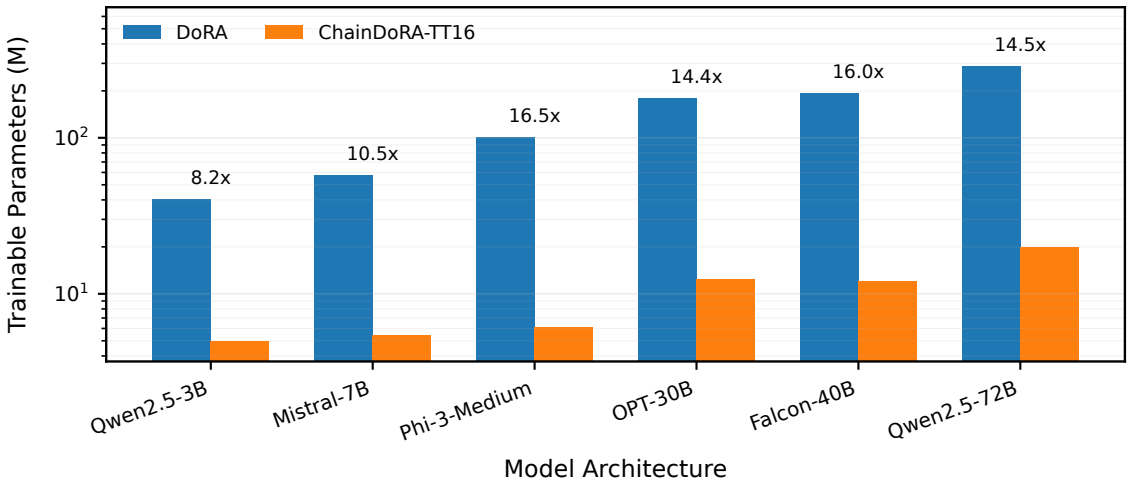


Figure 6: Trainable-parameter scaling of full-target DoRA and ChainDoRA-TT16 across representative language-model architectures.

the trainable parameters. Combining these two design dimensions produces the TT8-MLP configuration, which improves upon TT4-Full in both parameter count and accuracy. These observations motivate treating TT rank and target-module placement as complementary design variables rather than optimizing either independently.

Third, the configuration-level scaling experiment indicates that the parameter savings are not restricted to the LLaMA-7B dimensions. The full-target ChainDoRA construction remains substantially smaller than DoRA across architectures ranging from approximately 3B to 72B parameters. However, this experiment evaluates parameter counts only and should not be interpreted as evidence that the LLaMA-7B accuracy gains transfer directly to these other model families.

Several limitations remain. The downstream experiments are conducted using a single pretrained backbone and a fixed 15,119-example adaptation pool rather than exhaustive full-dataset fine-tuning. The reported conclusions should therefore be interpreted as evidence from a controlled parameter-efficient adaptation setting rather than as claims of state-of-the-art commonsense reasoning performance. In addition, the cross-architecture study is analytical and does not include model training. The primary baselines use matched optimization hyperparameters to isolate adapter parameterization; method-specific hyperparameter searches could change their absolute performance. Future work should evaluate ChainDoRA across additional model families, larger adaptation datasets, multiple random seeds, and broader reasoning and generation benchmarks. Further investigation of TT contraction kernels and training-time efficiency would also be valuable, since reductions in trainable parameter count do not necessarily translate directly into equivalent reductions in computational cost.

6 Conclusion

This work introduced *ChainDoRA*, a parameter-efficient adaptation framework that combines magnitude–direction weight decomposition with a connected Tensor-Train representation of the directional low-rank factors. Instead of directly optimizing the dense matrices used by LoRA and DoRA, ChainDoRA constructs $A_{\mathrm{TT}} \in \mathbb{R}^{r \times d_{\mathrm{in}}}$ and $B_{\mathrm{TT}} \in \mathbb{R}^{d_{\mathrm{out}} \times r}$ from contractions of a shared TT chain. The adapter rank r is retained as the boundary rank between the input- and output-side contractions, while an independent internal TT rank ρ controls the capacity and parameter cost of the structured representation.

Under a controlled 15,119-example adaptation setting with LLaMA-7B, ChainDoRA-TT16 achieved a seven-task average commonsense reasoning accuracy of 72.30%, compared with 69.88% for LoRA and 69.39% for DoRA. At the same adapter rank $r = 32$, ChainDoRA required only 5.35M trainable parameters, compared with 56.10M for LoRA and 56.98M for DoRA, corresponding to a 90.62% reduction relative to DoRA. The TT-rank ablation further demonstrated a monotonic capacity–performance trade-off across $\rho \in \{4, 8, 16\}$, while the adapter-placement study showed that selective MLP adaptation can retain much of the full QKV+MLP ChainDoRA performance with substantially fewer trainable parameters. Combining these two compression dimensions produced the TT8-MLP configuration, which achieved a more favorable parameter–accuracy operating point than TT4-Full.

The configuration-level scaling analysis further showed that the structural parameter savings of ChainDoRA persist across model architectures ranging from approximately 3B to 72B parameters, although downstream performance on these additional architectures remains to be evaluated. Overall, the results suggest that the dense di-

rectional factors used in conventional magnitude–direction adaptation can be replaced by a structured tensor representation while retaining strong downstream performance and substantially reducing the number of task-specific trainable parameters.

Future work will investigate ChainDoRA across additional language-model families, larger and more diverse adaptation datasets, multiple random seeds, and broader reasoning and generation tasks. Further optimization of TT contraction and normalization kernels may also improve training-time efficiency, since reductions in trainable parameter count do not necessarily translate directly into proportional reductions in computational cost.

References

[1] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799, 2019.

[2] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, pages 4582–4597, 2021. doi: 10.18653/v1/2021.acl-long.353.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=nZeVKeeFYf9`.

[4] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32100–32121. PMLR, 2024. URL `https://proceedings.mlr.press/v235/liu24bn.html`.

[5] Ivan V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, 2011. doi: 10.1137/090752286.

[6] Alexander Novikov, Dmitrii Podoprikhin, Anton Osokin, and Dmitry P. Vetrov. Tensorizing neural networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[7] Yifan Yang, Jiajun Zhou, Ngai Wong, and Zheng Zhang. LoRETTA: Low-rank economic tensor-train adaptation for ultra-low-parameter fine-tuning of large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3161–3176, Mexico City, Mexico, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.174.

[8] Afia Anjum, Maksim E. Eren, Ismael Boureima, Boian Alexandrov, and Manish Bhattarai. Tensor train low-rank approximation (TT-LoRA): Democratizing ai with accelerated llms. *arXiv preprint arXiv:2408.01008*, 2024.

[9] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations*, 2023.

[10] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, pages 10088–10115, 2023.

[11] Dawid Jan Kopiczko, Tijmen Blankevoort, and Yuki M. Asano. VeRA: Vector-based random matrix adaptation. In *International Conference on Learning Representations*, 2024.

[12] Xiaolin Hu, Xiang Cheng, Peiyu Liu, Wei Liu, Jian Luan, Bin Wang, and Yong Liu. DoTA: Weight-decomposed tensor adaptation for large language models. *arXiv preprint arXiv:2412.20891*, 2024.

[13] Jun Qi, Chen-Yu Liu, Sabato Marco Siniscalchi, Chao-Han Huck Yang, and Min-Hsiu Hsieh. Joint tensor-train parameterization for efficient and expressive low-rank adaptation. *arXiv preprint arXiv:2506.16456*, 2025.

[14] Alexandra Zelenin and Alexandra Zhuravlyova. Scaling DoRA: High-rank adaptation via factored norms and fused kernels. *arXiv preprint arXiv:2603.22276*, 2026. doi: 10.48550/arXiv.2603.22276.

[15] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[16] Zhiqiang Hu, Lei Wang, Yihuai Lan, Wanyu Xu, Ee-Peng Lim, Lidong Bing, Xing Xu, Soujanya Poria,

and Roy Lee. LLM-adapters: An adapter family for parameter-efficient fine-tuning of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5254–5276, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.319.

[17] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2924–2936, Minneapolis, Minnesota, 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1300.

[18] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439, 2020. doi: 10.1609/aaai.v34i05.6239.

[19] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. Social IQa: Commonsense reasoning about social interactions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4463–4473, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1454.

[20] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8732–8740, 2020. doi: 10.1609/aaai.v34i05.6399.

[21] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[22] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391, Brussels, Belgium, 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1260.

[23] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[24] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

[25] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

[26] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. OPT: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

[27] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, Daniele Mazzotta, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. The falcon series of open language models. *arXiv preprint arXiv:2311.16867*, 2023.